Praktikum 4: Ray Trace

In this fourth practical exercise, a lot has happened in comparison to the last exercise, especially in `vec3 Scene::traceRay(const Ray& ray, double IoR, int recDepth) const`. Unfortunately, the great developer did not manage to fully implement the raytracing because he was busy making an advent calendar. Complete the source code so that the raytracer can now at least render very simple images including reflection by reflective and refraction by translucent objects. Read the new comments in the source code or use the internet to understand how reflection and refraction as well as the Phong Lighting model should roughly work.

Good luck!

If you have any questions or problems, please write to us in Moodle!

Aufgabe 1 — Vectoren

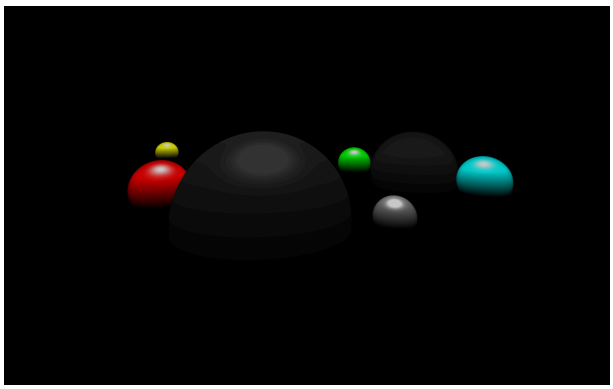


Figure 1: This is what you should see if you run the code without changing anything.

a) Open https://git.uni-due.de/vs.ude/objektorientierte-programmierung-cpp. Use git to *clone* the repository and import the project into 04_RayTrace/.

b) In `Vector3.cpp` you will find `vec3 vec3::reflection(const vec3 &normal) const`, which is used to calculate how a vector is reflected from a surface normal `normal`. The principle is quite simple, if you throw a tennis ball at a wall, it depends on how the wall is tilted to determine how the ball flies after the bounce. This is also how tennis works. In the commentary there is a hint on how to mathematically calculate a reflection. Once you've done it, you can return `//return local_color * l + reflection * r;` instead of the previous return value and you should see an image that looks like this:

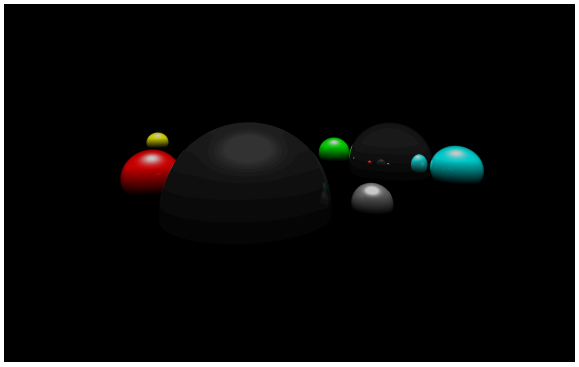


Figure 2: This is what you should see if you have implemented the reflection correctly. Note in particular the mirrored coloured spheres in the mirror ball on the right-hand side of the image. Feel free to play around with the values of the spheres in main.cpp to see how the effects change.

c) Fortunately, the developer has already implemented `std::optional<vec3> vec3::refraction(const vec3 &normal, double IORRatio) const`. It is somewhat more complicated than the reflection, as it is possible that the beam disappears in the material due to **Total Internal Refraction**. However, it has not been implemented that the **refraction** rays are also traced. With `std::optional<vec3> refractionDir = ray._direction.refraction(normal, intersection->_material.getIndexOfRefraction());` you get the direction in which the curvature ray points. Fortunately, you can use `Ray reflectionRay(intersectionPoint + normal * epsilon, ray._direction.reflection(normal));` as a guide to what a `refractionRay` should look like. Complete the code and comment `//return local_color * l + refraction * t;` in place of the previous return value to see if your source code works. If the refraction was implemented correctly, it will look like below:

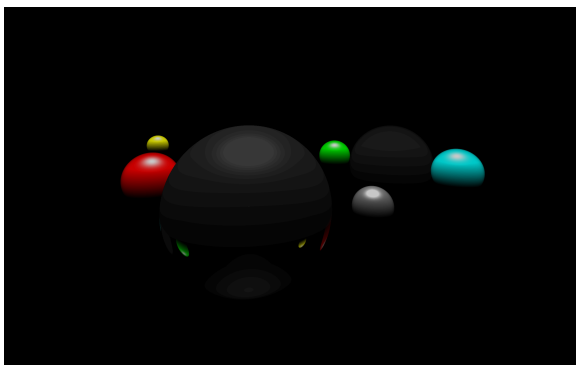


Figure 3: This is what you should see if the refractionRay has been implemented correctly. The spheres in the glass ball on the left are at the bottom, this is due to funny light physics.

If you have solved reflection and refraction, you can comment `//return local_color * l + reflection * r + refraction * t;` and see if your image looks exactly like below:

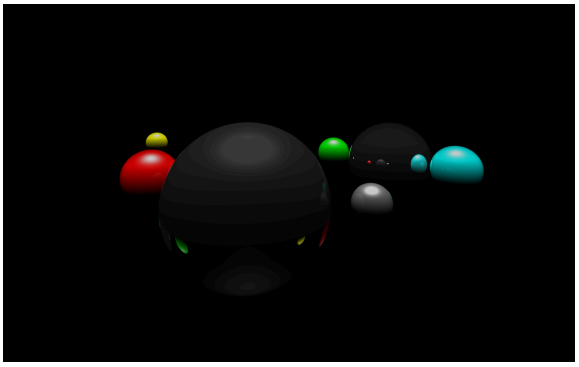


Figure 4: This is what you should see when you're done.

d) Note on submission: The submission should be a .zip archive containing only C++ source code and a file showing who the group members are with first name, surname and matriculation number.